

# SQL Azure with ASP Dot Net

By Don Schlichting

## Introduction

SQL Azure (SSDS - SQL Server Data Services) is a cloud database system offered by Microsoft. We interact with the SQL Azure services by either issuing statements to it through a command prompt or developing Dot Net applications. This article will introduce and demonstrate development of SQL Azure ASP Dot Net applications.

## Connecting

Connecting to SQL Azure is similar to connecting to a traditional SQL Server. A connection string is created then opened. To begin with SQL Azure, create an account from the Microsoft Azure web site. Once the account is created, log in and navigate to the Server Administration page. Select the master database (master is automatically created for us when the Azure account is created), and then select "Connection Strings". A popup will display the connection string for ADO Dot Net and ODBC. Copy the ADO Dot Net string. On the next tab, Firewall Settings, create a rule to allow your client machine access as shown below.



The IP address will be your outside connection address. The firewall tool will display the IP address you connected from. SQL Azure services are firewall protected, so each client requiring access will need a rule. If you're developing from a machine which receives a DHCP address from an internet service provider, then a new rule will need to be created each time a new address is received (unless you know the IP address range, then it can be specified here). Allow up to five minutes for the new firewall rule to go into effect.

Create a new Dot Net web site and aspx page from Visual Studio. In the Page Load section, add using statements for System.Data and System.Data.SqlClient just as we would for a traditional SQL Server web page. Save the connection string as a variable and pass it to the SqlConnection object as shown below:

```
protected void Page_Load(object sender, EventArgs e)
{
    string sConn = "Server=tcp:qdpapn8.database.windows.net;
    Database=master;
    User ID=dons;
```

```

        Password=mypassword;Trusted_Connection=False";

        SqlConnection objConn = new SqlConnection(sConn);
        objConn.Open();

        objConn.Close();

    }

```

The connection string ID and password were specified when the Azure account was first created. The server section specifies the fully qualified name. In this example, qdpapn8, is my personal unique server that SQL Azure auto generated for my account. My personal server name is appended to the public "database.windows.net" domain to create the Fully Qualified Name (FQN). Note the "tcp" prefix to the server name. SQL Azure listens on TCP port 1433, so our client workstation must allow outbound traffic on this port. The Trusted Connection will always be set to False, as Windows Logins are not supported on SQL Azure. Executing this web page should result in a blank Internet Explorer window. If an error is thrown, it will be displayed there instead.

## CREATE DATABASE

Now that we have a valid connection string, we'll create a test database. The TSQL syntax for SQL Azure is usually identical to traditional SQL Server. A full list of supported statements, limitations, and options is available from this Microsoft web site: [Transact SQL Reference SQL Azure Database](#). In this example, we'll pass a CREATE DATABASE statement to a SqlCommand object as shown below. Remember to add the System.Data and System.Data.SqlClient using statements to all examples in this article.

```

protected void Page_Load(object sender, EventArgs e)
{
    string sConn = "Server=tcp:qdpapn8.database.windows.net;Database=master;User
ID=dons;
                    Password=mypwd;Trusted_Connection=False";

    SqlConnection objConn = new SqlConnection(sConn);
    objConn.Open();

    string sSQL = "CREATE DATABASE TestDB";

    SqlCommand objCmd = new SqlCommand(sSQL, objConn);
    objCmd.CommandType = System.Data.CommandType.Text;
    objCmd.ExecuteNonQuery();

    objConn.Close();
}

```

The CREATE DATABASE statement was saved as a string and then passed as a Text Command statement. After executing this page, a new 1 GB database was created as shown in the Server Administration web page.

| Databases     |       | Firewall Settings |           |
|---------------|-------|-------------------|-----------|
| Database Name | Size  | Type              | Available |
| master        | 64 KB | 1 GB              | Yes       |
| TestDB        | 0 B   | 1 GB              | Yes       |

Currently, there are two different database sizes, 1 GB or 10 GB, each with a different pricing policy.

## CREATE TABLE

To create a Table in the new database, change the connection string specifying the new Database Name and change the statement to a typical CREATE TABLE as shown below. This example will create a two-column table.

```
protected void Page_Load(object sender, EventArgs e)
{
    string sConn = "Server=tcp:qdpapn8.database.windows.net;Database=TestDB;User
ID=dons;
                    Password=mypwd;Trusted_Connection=False";

    SqlConnection objConn = new SqlConnection(sConn);
    objConn.Open();

    string sSQL = "CREATE TABLE myTestTable
                  ( FirstName varchar(50) NOT NULL, LastName varchar(50) NULL,
                    CONSTRAINT PK_TestTable PRIMARY KEY (FirstName) )";

    SqlCommand objCmd = new SqlCommand(sSQL, objConn);
    objCmd.CommandType = System.Data.CommandType.Text;
    objCmd.ExecuteNonQuery();

    objConn.Close();
}
```

Note that the CREATE TABLE statement included a key, in this case a PRIMARY KEY. Heap tables (tables without an index) are not allowed in SQL Azure.

## INSERT

The syntax for inserting data into SQL Azure is identical to traditional SQL Server. Here we've modified the SQL string with an INSERT. The rest of the code, SqlConnection, SqlCommand, etc., are identical to the previous examples.

```
string sSQL = "INSERT INTO myTestTable (FirstName, LastName) VALUES ('bob', 'smith') ";
```

## SELECT

To receive the newly inserted row back, drag a GridView control onto a new page and modify the SQL string to a SELECT statement. Then we'll bind the GridView to the SqlCommand object and execute the Reader as shown below:

```
protected void Page_Load(object sender, EventArgs e)
{
    string sConn = "Server=tcp:qdpapn8.database.windows.net;Database=TestDB;User
ID=dons;
                    Password=mypwd;Trusted_Connection=False";

    SqlConnection objConn = new SqlConnection(sConn);
    objConn.Open();

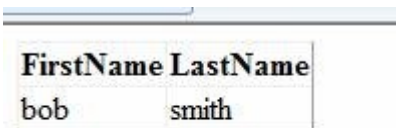
    string sSQL = "SELECT * FROM myTestTable";

    SqlCommand objCmd = new SqlCommand(sSQL, objConn);
    objCmd.CommandType = System.Data.CommandType.Text;

    myGridView.DataSource = objCmd.ExecuteReader();
    myGridView.DataBind();

    objConn.Close();
}
```

The result will be our one inserted row being returned as shown below:



| FirstName | LastName |
|-----------|----------|
| bob       | smith    |

## Stored Procedures

In my opinion, one of the best features of SQL Azure is the ability to create Stored Procedures in the cloud. This is a rare feature, most cloud databases do not support stored procedures and instead database code must be passed as a string. By using Stored Procedures, we create a clear separation between database code and application code. In addition, it's easy to share the same stored procedure between many web pages and at the same time we avoid SQL Injection attacks. The syntax for creating a Stored Procedure in the cloud is the same as traditional SQL Server. In this example, we'll simply modify the SQL string as shown below:

```
protected void Page_Load(object sender, EventArgs e)
{
    string sConn = "Server=tcp:qdpapn8.database.windows.net;Database=TestDB;User
ID=dons;
                    Password=mypassword;Trusted_Connection=False";

    SqlConnection objConn = new SqlConnection(sConn);
```

```

objConn.Open();

string sSQL = "CREATE PROCEDURE GetNames AS SELECT * FROM myTestTable";

SqlCommand objCmd = new SqlCommand(sSQL, objConn);
objCmd.CommandType = System.Data.CommandType.Text;

objCmd.ExecuteNonQuery();
objConn.Close();

}

```

To execute the Stored Procedure, we'll drag a GridView onto the form; bind that grid to the SqlCommand object, and then call the ExecuteReader as shown below.

```

protected void Page_Load(object sender, EventArgs e)
{
    string sConn = "Server=tcp:qdpapn8.database.windows.net;Database=TestDB;User
ID=dons;
                    Password=mypassword;Trusted_Connection=False";

    SqlConnection objConn = new SqlConnection(sConn);
    objConn.Open();

    SqlCommand objCmd = new SqlCommand("dbo.GetNames", objConn);
    objCmd.CommandType = System.Data.CommandType.StoredProcedure;

    myGridView.DataSource = objCmd.ExecuteReader();
    myGridView.DataBind();

    objConn.Close();
}

```

## Conclusion

Working with SQL Azure (SQL Server Data Services SSDS) leverages our existing TSQL and Dot Net skills. The TSQL for the cloud usually has the same syntax as a traditional SQL Server, and calling the cloud from a web page uses the same commands and readers as other familiar data sources. Having the ability to create relationships, use TSQL, and execute stored procedures in the cloud are strong benefits of SQL Azure.